

THEOREM

AMD MI300X-Optimized GPU Kernels

CDNA3-tuned Triton kernels for sub-quadratic sequence models



Why Theorem?



Sub-Quadratic Models Are Here

Mamba, Mamba-2, and gated DeltaNet replace attention with selective SSMS and delta-rule recurrences — far cheaper at long context, but kernel-starved on AMD hardware.



MI300X Is Underserved

The AMD Instinct MI300X packs 192 GB HBM3e and 304 CDNA3 compute units — massive throughput potential. Yet most open-source kernels are written for CUDA/A100, leaving performance on the table.



No Generality, Just Speed

Theorem doesn't chase portability. Every kernel knows exactly what hardware it runs on — gfx942, 64-lane wavefronts, Matrix Cores — and squeezes accordingly.

Hardware Target

AMD Instinct MI300X · CDNA3 (gfx942)

Property	Value
Device	AMD Instinct MI300X
Architecture	CDNA3 (gfx942)
Compute Units	304 CUs
Wavefront Width	64 lanes
Memory	~192 GB HBM3e
Stack	ROCm 7.x · HIP · Triton (AMD backend $\geq$ 3.1)

304

Compute Units

64

Lane Wavefronts

192

GB HBM3e

gfx942

GPU Target ID

Kernel Inventory

causal_conv1d

$$y[t, c] = \sum_k w[k, c] \cdot x[t-k, c]$$

USE Mamba / Mamba-2 depthwise 1D causal convolution

OPT 64-lane wavefront block sizing · LDS-resident weight tile · SiLU fused epilogue

chunk_fwd_h

$$S_{\{c+1\}} = G_c \cdot S_c + K_c^T V_c$$

USE Gated DeltaNet inter-chunk state recurrence (arXiv:2412.06464)

OPT State S pinned in LDS · tl.dot → Matrix Core MFMA · K/V pipelined via num_stages

chunk_fwd_o

$$O_c = (Q_c K_c^T \odot M) V_c + Q_c S_c$$

USE Chunkwise output: local causal attention + global state read

OPT Q tile shared in registers across two tl.dot blocks · causal mask compile-time · coalesced HBM3e state read

recompute_w_u

$$W = \beta \cdot (I - \text{tril}(KK^T)) \cdot \beta)^{-1}$$

USE WY-transform helpers for backward pass (avoids HBM spills)

OPT In-place lower-triangular solve unrolled at BT · recompute traded against HBM bandwidth

Four Core Optimization Principles

01

Wavefront-Aware Block Sizing

CDNA3 issues in 64-lane wavefronts. Every block size is a multiple of 64 — never 32. Mismatched shapes leave half the wavefront idle and burn LDS for no work.

02

LDS Pipelining via num_stages

Kernels with inner reduction loops set `num_stages` ≥ 2 so the next tile's HBM3e load overlaps the current Matrix Core dot. Too high $\rightarrow$ LDS pressure; too low $\rightarrow$ memory serialization.

03

Fused Dot-Accumulate on Matrix Cores

Reductions go through `tl.dot` with `acc= chaining`, lowered onto MFMA instructions in `fp16/bf16` with `fp32` accumulate. Hand-rolled `tl.sum` is only a sanity baseline.

04

Per-Shape Configuration Tuning

`BT`, `BK`, `BV`, `num_warps`, `num_stages`, `waves-per-EU` live in YAML. Sweep driver finds the best config per (batch, heads, seqlen, dim). No runtime autotune on the hot path.

Tooling & Repo Layout

TOOLING

PyTorch + ROCm

Host tensor management & reference implementations

Triton (AMD backend)

Kernel authoring → HIP-compatible binaries (≥ 3.1)

NumPy

Numerical reference utilities & offline analysis

PyYAML

Per-shape autotune configuration files

KEY ENTRY POINTS

eval.py

Correctness + microbenchmarks per kernel

run_sweep.py

Drives autotune sweeps across per-shape configs

scripts/run_amd.py

Orchestrates a full benchmark run on MI300X

scripts/monitor_gpu.sh

rocm-smi: power, clock, HBM occupancy traces

scripts/cpu_reference.py

NumPy oracles for bitwise-stable validation

Quick Start

```
# Clone & set up
git clone https://github.com/yhinai/Theorem.git
cd Theorem

# Create virtual environment
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Configure ROCm + Triton env vars
bash scripts/setup_env.sh

# Run correctness + microbench
python eval.py test kernels/causal_conv1d/
```

PRE-REQUISITES

HARDWARE

AMD Instinct MI300X

OS STACK

ROCm 7.x + HIP

PYTHON

PyTorch ROCm wheel (see requirements.txt --index-url)

TRITON

AMD backend ≥ 3.1 (upstream)

⚠ Use the ROCm PyTorch wheel, not the standard PyPI package

Benchmarking & Results

HOW TO RUN

```
# Lock clocks, flush cache, start GPU trace
bash scripts/monitor_gpu.sh results/gpu_trace.log &

# Benchmark all kernels
python scripts/run_amd.py --kernels all --out results/

# Sweep autotune configs for one kernel
python run_sweep.py --kernel chunk_fwd_o \
  --config kernels/chunk_fwd_o/configs.yaml
```

OUTPUT CSV COLUMNS

shape

mean_ms

p50_ms

p99_ms

gbps

tflops

METHODOLOGY

Warmup

Fixed warmup iterations before timing to steady GPU clocks

Clock Lock

Clocks pinned via rocm-smi during sweep to reduce variance

Cache Flush

L2 cache flushed between iterations for fair memory timing

YAML Config

Best (shape → config) mapping found offline, imported at load time

The mission:

**No generality.
No portability shims.
Just kernels that know
exactly what
hardware they
run on.**

THEOREM AT A GLANCE

4 kernels

causal_conv1d, chunk_fwd_h, chunk_fwd_o, recompute_w_u

AMD MI300X

CDNA3 gfx942, 192 GB HBM3e, 304 CUs

Triton (AMD)

upstream ≥ 3.1 , HIP backend, MFMA Matrix Cores

4 principles

wavefront sizing, LDS pipeline, fused dot, per-shape YAML

MIT License

github.com/yhinai/Theorem